UNIVERSIDADE DA CORUÑA

Grado en Inteligencia Artificial

Fundamentos de Procesamiento del Lenguaje Natural – Curso 2025-2026

Analizadores sintácticos de dependencias basados en transiciones

Preliminares: El objetivo de la práctica es familiarizarse con el análisis sintáctico de dependencias, así como entender e implementar un algoritmo de análisis sintáctico basado en transiciones. Con esto en mente, en esta práctica replicaremos una versión simplificada de un artículo clásico de NLP y uno de los primeros analizadores que hizo uso de redes neuronales (sencilla, un perceptrón multi-capa, en esta práctica nos centraremos más en los algoritmos de parsing y no tanto en la red neuronal): "A Fast and Accurate Dependency Parser using Neural Networks" (Chen y Manning, 2014).

Dada una serie de oraciones en texto plano, el objetivo de la práctica es obtener su representación sintáctica, tal y como se ilustra en la Figura 1.

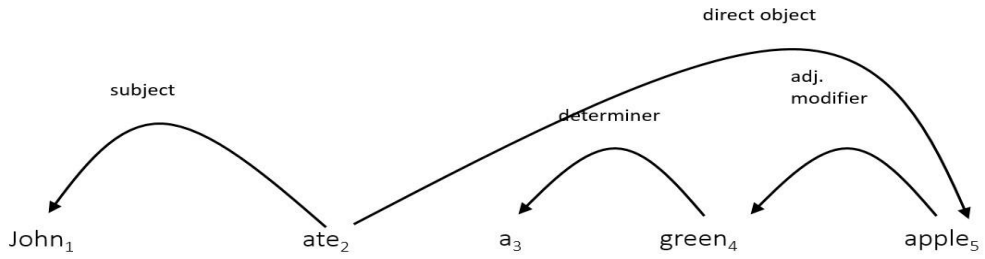

Figura 1. Ejemplo de una oración de entrada ('John ate a green apple') y su correspondiente estructura sintáctica de acuerdo a una gramática de dependencias.

Algoritmos de transiciones: Utilizaremos el algoritmo arc-eager. El conjunto de transiciones y precondiciones se describen en la Tabla 1. Para más información sobre el algoritmo arc-eager se recomienda consultar la `guia_p1.pdf`, incluida con este enunciado.

Extracción de *features*: En cada paso, para predecir cada transición, el modelo recibirá como entrada una secuencia de *features* que representan el estado en el que actualmente se encuentra el analizador. El modelo considerará como características de entrada elementos extraídos directamente del stack y del buffer, lo que se denomina como *features* de primer orden. Para representar cada estado, se usarán los dos primeros elementos del buffer y los dos primeros elementos en la cima del stack. Para cada uno de estos elementos extraeremos la propia forma de la palabra y la PoS tag universal. Toda esta información se concatenará y servirá de entrada al clasificador que predecirá la siguiente transición y que actualizará el estado del algoritmo.

Transición	Estado actual	Siguiente estado
LEFT-ARC$_l$	$(\sigma\|i, j\|\beta, A)$	$(\sigma, j\|\beta, A \cup \{(j, l, i)\})$
RIGHT-ARC$_l$	$(\sigma\|i, j\|\beta, A)$	$(\sigma\|i\|j, \beta, A\cup \{(i, l, j)\})$
REDUCE	$(\sigma\|i, \beta, A)$	(σ, β, A)
SHIFT	$(\sigma, i\|\beta, A)$	$(\sigma\|i, \beta, A)$

Precondiciones	
LEFT-ARC$_l$	$\neg[i = 0]$ $\neg\exists k\exists l\ [(k, l , i) \in A]$
RIGHT-ARC$_l$	$\neg\exists k\exists l\ [(k, l , j) \in A]$
REDUCE	$\exists k\exists l[(k, l, i) \in A]$

Estado inicial	Estado final
$(\sigma =[ROOT], \beta=sentence, \{\})$	$(\sigma, \beta=[], A)$

Tabla 1. Conjunto de transiciones del algoritmo arc-eager. Las acciones LEFT-ARC$_l$ y RIGHT-ARC$_l$ que crean un arco entre un par de elementos del buffer y del stack acompañan la predicción de la etiqueta de dependencia l, que conecta esos dos términos (por ejemplo, sujeto o modificador).

Arquitectura neuronal: Cada feature será mapeada a un vector continuo (*embedding*), y la concatenación de dichos vectores de características servirán de entrada al clasificador. La arquitectura básica del modelo estará compuesta por una feed-forward network de una o más capas, a elección del estudiante, seguida de dos capas de salida con una función softmax: una que indique la siguiente acción a realizar y otra que clasifique el tipo de dependencia al crear un arco. La Figura 2 ilustra gráficamente la arquitectura a alto nivel, empleando como características de entrada las palabras y PoS tags de los dos elementos en la cima de la stack, y de los dos primeros elementos del buffer. Se dispone de libertad para definir los hiperparámetros de la red, optimizadores, ampliar el número de características de entrada respecto a la especificadas en la sección "*Extracción de features*", etc.

Treebank: Para esta práctica, deberá entrenarse y evaluarse el modelo sobre el corpus UD_English-ParTUT cuyos conjuntos de entrenamiento, desarrollo y test se incluyen con los materiales de esta práctica (ya preprocesados). Estos están representados en formato CoNLL-U (más detalles sobre el formato CoNLL-U con el que se representan árboles de dependencias pueden ser encontrados en https://universaldependencies.org/format.html). El dataset English-ParTUT contiene algunas oraciones no proyectivas (oraciones con arcos que se cruzan), que no pueden ser modeladas por el algoritmo arc-eager. En la práctica ya se da implementado todo lo relativo a la lectura y escritura de ficheros CoNLLU, incluyendo la eliminación de estas oraciones no proyectivas.

Tareas: Las funcionalidades que deben ser soportadas son:

1. **Entrenar un modelo,** y verificar que el modelo converge en el conjunto de desarrollo.
2. **Generación.** Una vez los modelos han sido entrenados, deberá implementarse una función que prediga los árboles de dependencias para el conjunto de test.
3. **Evaluación** Para reportar el rendimiento de los modelos sobre dicho conjunto, se emplearán las métricas estándar de evaluación para el análisis sintáctico de dependencias: *labelled* (LAS) y *unlabeled attachment scores* (UAS). Para ello, debe usarse el script de evaluación que puede

encontrarse en: https://universaldependencies.org/conll18/conll18_ud_eval.py que recibe como parámetros de entrada el fichero CoNLL-U gold y el fichero CoNLL-U predicho.

Adicionalmente, se valorarán experimentos y variantes adicionales que aporten valor al análisis, quedando a criterio del alumno su elección y justificación.

El fichero `guia_p1.pdf` contiene información más detallada sobre cómo llevar a cabo la práctica.

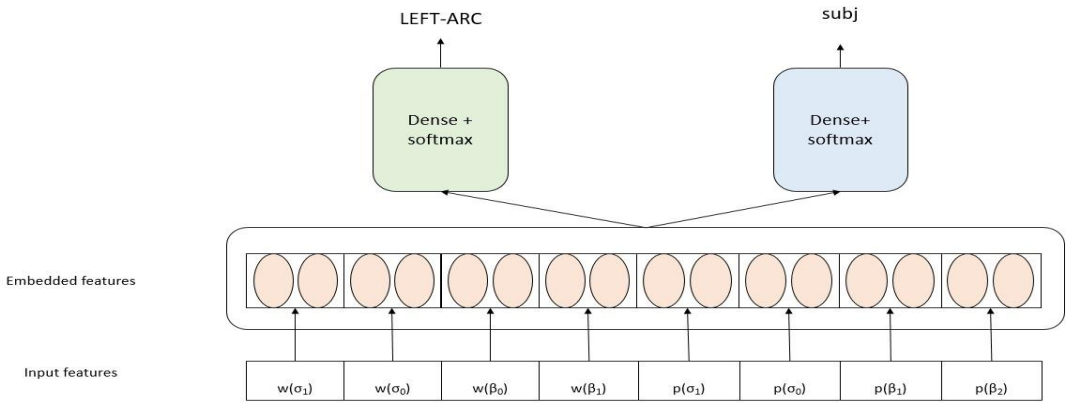

Figura 2. Arquitectura de alto nivel del modelo, usando los dos elementos en la cima de la pila y los dos primeros elementos del buffer. En el ejemplo, W es una función que recupera la palabra de un elemento determinado, y P es una función que recupera la etiqueta morfológica de dicho token.

Entrega

La tarea deberá incluir un breve manual de usuario que describa cómo ejecutar el código para el entrenamiento, la evaluación y la generación de árboles. La entrega podrá realizarse mediante un script de Python ejecutable o un notebook. Asimismo, se deberá incorporar una breve discusión de las decisiones de diseño adoptadas y de los resultados obtenidos por el modelo. La extensión máxima del documento será de 5 páginas.

- Las prácticas se realizarán con Python y Keras.

- La tarea se realizará en grupos de 2 personas.

- **El plazo de entrega en el campus virtual finaliza el lunes 11 de mayo a las 23:59 horas.** Sólo un miembro del grupo entregará el trabajo. Los trabajos entregados fuera de plazo se considerarán suspensos.